Ozak.ai - TokenPresale

Security Assessment

CertiK Assessed on Jun 16th, 2025



CertiK Assessed on Jun 16th, 2025

Ozak.ai - TokenPresale

The security assessment was prepared by CertiK, the leader in Web3.0 security.

Executive Summary

TYPES	ECOSYSTEM	METHODS
presale	EVM Compatible	Manual Review, Static Analysis

LANGUAGE	TIMELINE	KEY COMPONENTS
Solidity	Delivered on 06/16/2025	N/A

CODEBASE

Private

View All in Codebase Page

Vulnerability Summary



18	5	1	12	0
Total Findings	Resolved	Partially Resolved	Acknowledged	Declined

2	Centralization	2 Acknowledged	Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.
0	Critical		Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.
0	Major		Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.
4	Medium	3 Resolved, 1 Acknowledged	Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.
3	Minor	2 Resolved, 1 Partially Resolved	Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.
9	Informational	9 Acknowledged	Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | OZAK.AI - TOKENPRESALE

Summary

Executive Summary

Vulnerability Summary

Codebase

Audit Scope

Approach & Methods

Review Notes

Dependencies

Third Party Dependencies

Out Of Scope Dependencies

Recommendations

Findings

OZT-02 : Centralization Risks in TokenPresale.sol

OZT-03 : Centralized Control of Contract Upgrade

OZT-04 : `initialize()` Is Unprotected

OZT-05 : Oracle Price Is Not Checked

OZT-06 : Referrer Tokens Not Accounted For In Allocation

OZT-07 : Contract Only Supports Whitelist Tokens With 6 Decimals

OZT-08 : Missing Zero Address Validation

OZT-09 : `endTime` Not Updated

OZT-10 : Missing Validations

OZT-11 : Array missing `pop` function

OZT-12 : Comparison to Boolean Constant

OZT-13 : Users May Receive No Tokens If `minPurchase` Is Not Set Large Enough

OZT-14 : Specific Imports Are Not Used

OZT-15 : Typos And Inconsistencies

OZT-16 : Commented-Out Code

OZT-17 : Usage of Magic Numbers

OZT-18 : Rate Must Be Carefully Configured If Allocated Token Decimal Is Not 18

OZT-19 : Unlocked Compiler Version

Optimizations

OZT-01 : Unnecessary Events

Appendix

Disclaimer

CODEBASE | OZAK.AI - TOKENPRESALE

Repository

Private

AUDIT SCOPE | OZAK.AI - TOKENPRESALE

1 file audited ● 1 file with Acknowledged findings

ID	Repo	File	SHA256 Checksum
● TPO	Ozak-AI/presale-contracts	contracts/TokenPresale.sol	28236c476d9310a9280de2c1466c472f3224ab5a8b7e361977174bd977fdbf63

APPROACH & METHODS | OZAK.AI - TOKENPRESALE

This report has been prepared for Ozak.ai to discover issues and vulnerabilities in the source code of the Ozak.ai - TokenPresale project as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Manual Review and Static Analysis techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.

The security assessment resulted in findings that ranged from critical to informational. We recommend addressing these findings to ensure a high level of security standards and industry practices. We suggest recommendations that could better serve the project from the security perspective:

- Testing the smart contracts against both common and uncommon attack vectors;
- Enhance general coding practices for better structures of source codes;
- Add enough unit tests to cover the possible use cases;
- Provide more comments per each function for readability, especially contracts that are verified in public;
- Provide more transparency on privileged activities once the protocol is live.

REVIEW NOTES | OZAK.AI - TOKENPRESALE

Dependencies

Third Party Dependencies

The protocol is serving as the underlying entity to interact with third party protocols. The third parties that the contracts interact with are:

- ERC20 Tokens

The scope of the audit treats third party entities as black boxes and assumes their functional correctness. However, in the real world, third parties can be compromised and this may lead to lost or stolen assets. Moreover, updates to the state of a project contract that are dependent on the read of the state of external third party contracts may make the project vulnerable to read-only reentrancy. In addition, upgrades of third parties can possibly create severe impacts, such as increasing fees of third parties, migrating to new LP pools, etc.

Out Of Scope Dependencies

The protocol is serving as the underlying entity to interact with out-of-scope dependencies. The out-of-scope dependencies that the contracts interact with are:

- Treasury

The scope of the audit treats out-of-scope dependencies as black boxes and assumes their functional correctness.

Recommendations

We recommend constantly monitoring the third parties involved to mitigate any side effects that may occur when unexpected changes are introduced, as well as vetting any third party contracts used to ensure no external calls can be made before updates to its state. Additionally, we recommend all out-of-scope dependencies are carefully vetted to ensure they function as intended.

FINDINGS | OZAK.AI - TOKENPRESALE



18	0	2	0	4	3	9
Total Findings	Critical	Centralization	Major	Medium	Minor	Informational

This report has been prepared to discover issues and vulnerabilities for Ozak.ai - TokenPresale. Through this audit, we have uncovered 18 issues ranging from different severity levels. Utilizing the techniques of Manual Review & Static Analysis to complement rigorous manual code reviews, we discovered the following findings:

ID	Title	Category	Severity	Status
OZT-02	**Centralization Risks In TokenPresale.Sol**	**Centralization**	**Centralization**	**Acknowledged**
OZT-03	**Centralized Control Of Contract Upgrade**	**Centralization**	**Centralization**	**Acknowledged**
OZT-04	`initialize()` Is Unprotected	Logical Issue	Medium	Resolved
OZT-05	Oracle Price Is Not Checked	Logical Issue	Medium	Resolved
OZT-06	Referrer Tokens Not Accounted For In Allocation	Logical Issue	Medium	Resolved
OZT-07	Contract Only Supports Whitelist Tokens With 6 Decimals	Logical Issue	Medium	Acknowledged
OZT-08	Missing Zero Address Validation	Volatile Code	Minor	Resolved
OZT-09	`endTime` Not Updated	Code Optimization	Minor	Resolved
OZT-10	Missing Validations	Logical Issue	Minor	Partially Resolved
OZT-11	Array Missing `pop` Function	Volatile Code	Informational	Acknowledged
OZT-12	Comparison To Boolean Constant	Coding Style	Informational	Acknowledged

ID	Title	Category	Severity	Status
OZT-13	Users May Receive No Tokens If `minPurchase` Is Not Set Large Enough	Logical Issue	Informational	Acknowledged
OZT-14	Specific Imports Are Not Used	Coding Style	Informational	Acknowledged
OZT-15	Typos And Inconsistencies	Inconsistency	Informational	Acknowledged
OZT-16	Commented-Out Code	Coding Style	Informational	Acknowledged
OZT-17	Usage Of Magic Numbers	Coding Style	Informational	Acknowledged
OZT-18	Rate Must Be Carefully Configured If Allocated Token Decimal Is Not 18	Logical Issue	Informational	Acknowledged
OZT-19	Unlocked Compiler Version	Coding Style	Informational	Acknowledged

OZT-02 | CENTRALIZATION RISKS IN TOKENPRESALE.SOL

Category	Severity	Location	Status
Centralization	**Centralization**	**contracts/TokenPresale.sol (Base): 167, 202, 212, 224, 233, 241, 250, 265, 275, 289, 309, 321, 335, 356, 430, 512, 582, 653, 666, 680, 763, 797**	**Acknowledged**

Description

In the contract `TokenPresale`, the role `_owner` has authority over the functions shown in the diagram below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and do the following:

- End the current presale stage
- Update the token address
- Update the presale stage parameters
- Withdraw any native or ERC20 tokens held by the contract to the treasury
- Initialize the next presale stage
- Update minimum and maximum purchase limits
- Whitelist payment tokens
- Update the vesting duration
- Release claims
- Start a new presale stage
- Finalize the presale event
- Update the referrer bonus percentage
- Pause the presale
- Update the vesting percentage
- Update the treasury address
- Update the cliff duration
- Release the referrer bonus
- Delist payment tokens

It should be noted that most of these operations can be performed at any time and if executed at the wrong moment may cause undesired behavior in the protocol.


Authenticated Role
_owner
Function
endPresaleStage
State Variables
stages
currentStageId
Function
finalisePresale
State Variables
vestingStartTime
presaleEnded
Function
updateTokenAddress
State Variables
token
Function
updatePresaleStageParameters
State Variables
stages
currentStageId
External Calls
tokenContract.balanceOf
Function
withdrawAccidentalFunds
External Calls
tokenContract.safeTransfer
External Calls
treasury.transfer
Function
initNextPresaleStage
State Variables
stages
currentStageId
Internal Calls
Stage
Function
updateMinMaxPurchase
State Variables
minPurchase
maxPurchase
Function
whitelistPaymentTokens
State Variables
whitelistedTokens
External Calls
tokenList.push
Function
updateVestingDuration
State Variables
vestingDuration
Function
releaseClaim
State Variables
claimReleased
Function
resumePresale
Internal Calls
_unpause
Function
updateReferrerBonusPercentage
State Variables
referrerBonusPercentage
Function
pausePresale
Internal Calls
_pause
Function
updateVestingPercentage
State Variables
vestingPercentage
Function
updateTreasury
State Variables
treasury
Function
updateCliffDuration
State Variables
cliffDuration
Function
releaseReferrerBonus
State Variables
referrerBonusReleased
Function
delistPaymentTokens
State Variables
whitelistedTokens

Furthermore, it is up to a centralized entity to ensure that the contract holds enough tokens for users to claim.

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (⅔, ⅗) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
  AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
  AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
  OR
- Remove the risky functionality.

Alleviation

[Ozak.ai, 06/12/2025]: "Issue Acknowledged. We're already using a Multi-sig with a high number of signers. We believe a timelock right now will kinder our speed to respond to ending and starting stages, but will incorporate Timelock mechanisms in the future. All the signers and multisig address is available onchain for everyone to view."

[CertiK, 06/13/2025]: After deployment please share the multi-sig address and transaction transferring ownership so that we can update this finding accordingly. Until then we leave this finding *Acknowledged*.

OZT-03 | CENTRALIZED CONTROL OF CONTRACT UPGRADE

Category	Severity	Location	Status
Centralization	**Centralization**	**contracts/TokenPresale.sol (Base): 33**	**Acknowledged**

Description

The `TokenPresale` contract is upgradeable and the `admin` of the proxy can change the implementation contract that the proxy points to.

Any compromise to the `admin` account may allow a hacker to take advantage of this authority and change the implementation contract which is pointed by proxy and therefore execute potential malicious functionality in the implementation contract.

Recommendation

We recommend that the team make efforts to restrict access to the admin of the proxy contract. A strategy of combining a time-lock and a multi-signature (⅔, ⅗) wallet can be used to prevent a single point of failure due to a private key compromise. In addition, the team should be transparent and notify the community in advance whenever they plan to migrate to a new implementation contract.

Here are some feasible short-term and long-term suggestions that would mitigate the potential risk to a different level and suggestions that would permanently fully resolve the risk.

Short Term:

A combination of a time-lock and a multi signature (⅔, ⅗) wallet mitigate the risk by delaying the sensitive operation and avoiding a single point of key management failure.

- A time-lock with reasonable latency, such as 48 hours, for awareness of privileged operations;
  AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to a private key compromised;
  AND
- A medium/blog link for sharing the time-lock contract and multi-signers addresses information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.

- Provide a link to the **medium/blog** with all of the above information included.

Long Term:

A combination of a time-lock on the contract upgrade operation and a DAO for controlling the upgrade operation mitigate the contract upgrade risk by applying transparency and decentralization.

- A time-lock with reasonable latency, such as 48 hours, for community awareness of privileged operations;
  AND
- Introduction of a DAO, governance, or voting module to increase decentralization, transparency, and user involvement;
  AND
- A medium/blog link for sharing the time-lock contract, multi-signers addresses, and DAO information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.
- Provide a link to the **medium/blog** with all of the above information included.

Permanent:

Renouncing ownership of the `admin` account or removing the upgrade functionality can *fully* resolve the risk.

- Renounce the ownership and never claim back the privileged role;
  OR
- Remove the risky functionality.

Note: we recommend the project team consider the long-term solution or the permanent solution. The project team shall make a decision based on the current state of their project, timeline, and project resources.

Alleviation

[Ozak.ai, 06/12/2025]: "Issue Acknowledged. We're already using a Multi-sig with a high number of signers. We believe a timelock right now will kinder our speed to respond to ending and starting stages, but will incorporate Timelock mechanisms in the future. All the signers and multisig address is available onchain for everyone to view."

[CertiK, 06/13/2025]: After deployment please share the multi-sig address and transaction transferring the proxy admin so that we can update this finding accordingly. Until then we leave this finding *Acknowledged*.

OZT-04 | `initialize()` IS UNPROTECTED

Category	Severity	Location	Status
Logical Issue	Medium	contracts/TokenPresale.sol (Base): 138; contracts/USDTToken.sol (Update1): 82, 110, 111	Resolved

Description

The `Token` logic contract does not protect the initializer. An attacker can front-run the `initialize` call and assume ownership of the logic contract. Once in control, the attacker can perform privileged operations, misleading users into believing that they are interacting with the legitimate owner of the upgradeable contract.

Recommendation

We recommend adding

```
/// @custom:oz-upgrades-unsafe-allow constructor
constructor() initializer {...}
```

OR

```
/// @custom:oz-upgrades-unsafe-allow constructor
constructor() {
  ...
  _disableInitializers();
}
```

The addition will prevent the function `initialize()` from being called directly in the implementation contract, but the proxy will still be able to `initialize()` its storage variables.

Alleviation

[CertiK, 06/13/2025]: The team heeded the advice and resolved the issue by disabling the initializer in the constructor in commit c0c2fdfbb9ac88deeed7c74bca1329d12a0a028a

OZT-05 | ORACLE PRICE IS NOT CHECKED

Category	Severity	Location	Status
Logical Issue	Medium	contracts/TokenPresale.sol (Base): 342~348	Resolved

Description

When fetching the price from chainlinks data feed it does not check for staleness or that the value is greater than 0.

If the feed stops functioning and the price becomes stale, then this can result in an incorrect price being used during purchases. Furthermore, if the feed provides a negative value, it will be cast as a uint256 resulting in an incorrect price being used during the purchase. In addition, if a value of 0 is returned, it will can either cause buys to revert or allow users to purchase 0 tokens.

Recommendation

We recommend adding a check for staleness and that the returned price is greater than 0.

Alleviation

[CertiK, 06/13/2025]: The client added checks that the price is greater than zero and that it is not more than 24 hours stale. We leave this finding pending so the following comments can be seen.

The staleness check should be based on the heartbeat of the feed. For example Chainlink's ETH/USD feed has a heartbeat of 1 hour, so it could be considered stale after 1 hour. We recommend ensuring the staleness check corresponds to the heartbeat of the feed that will be used.

[Ozak.ai, 06/16/2025]: Issue Acknowledged. Added fix to check for consider stale data after 1 hours 5 minutes. Extra 5 minutes to just allow for a buffer time incase of high network traffic or other anomalies. New Fix is in commit 288a901a74e4613a66df8052771788abb0569482.

OZT-06 REFERRER TOKENS NOT ACCOUNTED FOR IN ALLOCATION

Category	Severity	Location	Status
Logical Issue	Medium	contracts/TokenPresale.sol (Base): 377~380	Resolved

Description

When purchasing tokens any tokens given for referrals is not accounted for in the stage allocation. This can result in more tokens being allocated than should be during a stage. In the worst case scenario, this can result in users being unable to claim tokens they are entitled to as the contract will not hold enough tokens for all the allocations and rewards.

Recommendation

We recommend ensuring the referral rewards do not allocate more tokens than should be allocated during a stage.

Alleviation

[Ozak.ai, 06/12/2025]: "Tokens allocated for referrals are separate from the tokens allocated per stage. This is how the logic is supposed to work."

[CertiK, 06/13/2025]: Considering this is the intended design we mark this finding *Resolved*. However, we recommend careful setting and tracking of the referral reward percentage to ensure that there are enough tokens to be allocated for the referral rewards.

OZT-07 CONTRACT ONLY SUPPORTS WHITELIST TOKENS WITH 6 DECIMALS

Category	Severity	Location	Status
Logical Issue	Medium	contracts/TokenPresale.sol (Base): 376, 452, 523~525, 532, 596~598, 605	Acknowledged

Description

The contract can only support USD stablecoins that have 6 decimals. This is because all tokens use the same `stage.rate` and the `usdAmount` calculated in the `buyTokensETH()` function has 6 decimals.

If a token with different decimals is supported, then it will use the `stage.rate` and cause an incorrect amount of tokens to be calculated.

Scenario

Assume that tokenA has 6 decimals and tokenB has 18 decimals and they are both supported. Furthermore assume the `stage.rate = 1000` that is 1 token is 0.001 USD assuming 6 decimals.

Then `buyTokens(tokenA, 10e6, address(0))` allocates the user `(10e6 * 1 ether) / 1000 = 10_000e18` tokens. That is a user pays 10 USD for 10_000e18 allocated tokens.

However `buyTokens(tokenB, 10e18, address(0))` allocates the user `(10e18 * 1 ether) / 1000 = 10_000e30` tokens. That is a user pays 10 USD for 10_000e30 allocated tokens receiving significantly more.

Recommendation

We recommend refactoring the code to allow for whitelisted tokens with differing decimals, or ensure that only tokens with 6 decimals are whitelisted.

Alleviation

[CertiK, 06/13/2025]: The team refactored the code in commit c0c2fdfbb9ac88deeed7c74bca1329d12a0a028a. However, we leave this finding pending to ensure the following comments are seen.

The current adjustment fixes the `tokenAmount` calculation, however, the check for the min and max USD amounts still does not handle differing decimals.

```
require(
            usdAmount >= minPurchase && usdAmount <= maxPurchase,
            "Amount out of range"
        );
```

Furthermore, the accounting often uses the input `usdAmount` which significantly differs based on the decimals of the token.

One potential solution is to only allow tokens with decimals less than or equal to 18 and to normalize the input amount to 18 decimals prior to doing any checks or accounting. Note that the actual transfer of the token should still use the unnormalized input.

Furthermore, the logic assumes that these will only be USD stablecoins and assume that they will always be pegged to 1 USD of value. We assume that only well known stable coins that do not take a fee on transfer will be whitelisted. Note that if a stable coin that takes a fee on transfer is supported then the protocol should be adjusted to track the actual amount of tokens received. Furthermore, if a stablecoin depegs the protocol will still assume it is worth 1 USD of value, if this is a concern we recommend the use of an oracle for the stablecoin's USD value.

[Ozak.ai, 06/16/2025]: "Issue Acknowledged, we'll setup internal checks to ensure we use only 6 Decimal USD Stablecoins like USDT and USDC. We will not be adding other tokens which are not 6 decimals and USD stablecoins."

OZT-08 | MISSING ZERO ADDRESS VALIDATION

Category	Severity	Location	Status
Volatile Code	Minor	contracts/TokenPresale.sol (Base): 156	Resolved

Description

The cited address input is missing a check that it is not `address(0)` .

Recommendation

We recommend adding a check the passed-in address is not `address(0)` to prevent unexpected errors.

Alleviation

[CertiK, 06/13/2025]: The team heeded the advice and resolved the issue by adding zero address validations in commit c0c2fdfbb9ac88deeed7c74bca1329d12a0a028a.

OZT-09 | `endTime` NOT UPDATED

Category	Severity	Location	Status
Code Optimization	Minor	contracts/TokenPresale.sol (Base): 190	Resolved

Description

The struct `Stage` has the parameter `endTime` to specify when a stage was ended. However, this parameter is set to zero and never updated, so that it does not give the stages end time.

Recommendation

We recommend updating the `endtime` when the stage is ended by calling `endPresaleStage()`.

Alleviation

[CertiK, 06/13/2025]: The team heeded the advice and resolved the issue by updating the end time in commit c0c2fdfbb9ac88deeed7c74bca1329d12a0a028a.

OZT-10 | MISSING VALIDATIONS

Category	Severity	Location	Status
Logical Issue	Minor	contracts/TokenPresale.sol (Base): 152~159, 198~206	Partially Resolved

Description

- `initialize()` does not validate its inputs which should be subjected to the same validations present in their update functions.
- `updateTokenAddress()` should only be called once. If it is called multiple times, then users may claim different tokens depending on when they are claimed.
- `updateReferrerBonusPercentage()` should not be called in the middle of a stage. If it is, then referrer rewards may differ within the same stage.
- `updateVestingDuration()` should not be called once vesting has started. Otherwise, if it is changed in the middle of vesting it can cause undesired behavior.
- `updateCliffDuration()` should not be called once vesting has started. Otherwise, if it is changed in the middle of vesting it can cause undesired behavior. In addition, it should be checked that the new cliff is less than the vesting duration.
- `updatePresaleStageParameters()` should only be called with an input allocation that is greater than the current amount of tokens sold and rewarded. In addition, the input rate should be checked to be greater than 0 and can be checked to be less than some predetermined maximum. Furthermore, it should only be called if `currentStageId > 0` as the stageID of 0 is not used.
- `updateVestingPercentage()` should not be called once vesting has started. Otherwise, if it is changed in the middle of vesting it can cause undesired behavior.
- `finalisePresale()` should not be called before at least one stage is initiated.

Recommendation

We recommend adding the validations mentioned above.

Alleviation

[Ozak.ai, 06/12/2025]:

Since all of these are owner callable function, we'll ensure an internal system to mitigate most of these issues. Nonetheless, some of these issues are fixed in this commit hash https://github.com/Ozak-AI/presale-contracts/commit/c0c2fdfbb9ac88deeed7c74bca1329d12a0a028a

[CertiK, 06/13/2025]: The client partially resolved the issue by adding some of the validations in commit

c0c2fdfbb9ac88deeed7c74bca1329d12a0a028a.

OZT-11 | ARRAY MISSING `pop` FUNCTION

Category	Severity	Location	Status
Volatile Code	Informational	contracts/TokenPresale.sol (Base): 62	Acknowledged

Description

Arrays without the pop operation in Solidity can lead to inefficient memory management and increase the likelihood of out-of-gas errors.

Recommendation

Consider adding functionality to remove elements from the array to prevent it from becoming too large over the lifetime of the contract.

Alleviation

[Ozak.ai, 06/16/2025]: "Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement."

OZT-12 | COMPARISON TO BOOLEAN CONSTANT

Category	Severity	Location	Status
Coding Style	Informational	contracts/TokenPresale.sol (Base): 106~109, 111~117, 167~196, 666~674, 813~829	Acknowledged

Description

Boolean constants can be used directly and do not need to be compared to true or false.

```
112         require(
113             stages[currentStageId].isActive == true,
114             "Current Stage not active"
115         );
```

```
176             require(
177                 stages[currentStageId].isActive == false,
178                 "Current stage is still active"
179             );
```

```
816             if (whitelistedTokens[tokenList[i]] == true) {
```

```
668         if (stages[currentStageId].isActive == true) {
```

```
823             if (whitelistedTokens[tokenList[k]] == true) {
```

```
107         require(presaleEnded != true, "Presale not active");
```

Recommendation

We recommend removing the comparison to true and false and instead using the boolean itself for true and its negation for false.

Alleviation

[Ozak.ai, 06/16/2025]: "Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement."

OZT-13 USERS MAY RECEIVE NO TOKENS IF `minPurchase` IS NOT SET LARGE ENOUGH

Category	Severity	Location	Status
Logical Issue	Informational	contracts/TokenPresale.sol (Base): 376, 452, 532, 605	Acknowledged

Description

If the `minPurchase` amount is not large enough considering the `stage.rate` to ensure that the calculated `tokenAmount` is nonzero, then a user may receive 0 tokens when paying a nonzero amount.

Recommendation

We recommend adding a check that the `tokenAmount` is above a minimum threshold to ensure that users will receive tokens from their purchase.

Alleviation

[Ozak.ai, 06/16/2025]: "Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement."

OZT-14 | SPECIFIC IMPORTS ARE NOT USED

Category	Severity	Location	Status
Coding Style	Informational	contracts/TokenPresale.sol (Base): 8~14	Acknowledged

Description

Specific imports are recommended to be used to avoid unpredictable pollution of the namespace.

Recommendation

We recommend using specific imports.

Alleviation

[Ozak.ai, 06/16/2025]: "Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement."

OZT-15 | TYPOS AND INCONSISTENCIES

Category	Severity	Location	Status
Inconsistency	Informational	contracts/TokenPresale.sol (Base): 405, 481, 666, 687, 727	Acknowledged

Description

- The comment for `elapsedTime` in the function `claimableTokens()` states "Calculate elapsed time since cliff end", however, it calculates the time since vesting started.
- The comments above the function `claimTokens()` state "Users can claim tokens only after the presale is finalized and the cliff duration has passed", however, users can claim tokens only after the presale is finalized and claims are released. Users may claim their `initialReleaseTokens` prior to the cliff.
- In the function `buyTokens()` it states "Transfer before state changes", however, it transfers after the state changes.
- The function `finalisePresale` has a typo and should be `finalizePresale()`.

Recommendation

We recommend fixing the typos and inconsistencies mentioned above.

Alleviation

[Ozak.ai, 06/16/2025]: "Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement."

OZT-16 | COMMENTED-OUT CODE

Category	Severity	Location	Status
Coding Style	Informational	contracts/TokenPresale.sol (Base): 723	Acknowledged

Description

The cited lines contain commented code that can be removed from the codebase.

Recommendation

We recommend either implementing commented-out code or removing it, as this will enhance the contract's readability and scalability.

Alleviation

[Ozak.ai, 06/16/2025]: "Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement."

OZT-17 | USAGE OF MAGIC NUMBERS

Category	Severity	Location	Status
Coding Style	Informational	contracts/TokenPresale.sol (Base): 215, 397, 397, 473, 557, 630, 630, 719, 719	Acknowledged

Description

Different magic numbers are used directly in the code.

Recommendation

We recommend declaring constants to improve code maintainability and readability. For example, `BONUS_FEE_DENOMINATOR = 10000`.

Alleviation

[Ozak.ai, 06/16/2025]: "Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement."

OZT-18 RATE MUST BE CAREFULLY CONFIGURED IF ALLOCATED TOKEN DECIMAL IS NOT 18

Category	Severity	Location	Status
Logical Issue	Informational	contracts/TokenPresale.sol (Base): 376, 452, 532, 605	Acknowledged

Description

The contract calculates the amount of allocated token as

```
uint256 tokenAmount = (usdAmount * 1 ether) / stage.rate;
```

Where the `stage.rate` is the USD amount needed to receive 1 ether of the the allocated token. This seems intended to be the USD amount needed to receive 1 of the allocated token assuming it has 18 decimals.

While the rate can be configured to handle allocated tokens with decimals other than 18, it may be set to an incorrect value if it is wrongly assumed that the rate should be the USD amount needed to receive 1 of the allocated token if the decimals are other than 18. For example if the allocated token has 6 decimals, then the rate should be the USD amount needed to receive 1e12 of the allocated token.

Recommendation

We recommend ensuring the rate is properly configured based on the decimals of the allocated token.

Alleviation

[Ozak.ai, 06/11/2025]: "Issue Acknowledged, Currently the plan is to have the Allocated token to have 18 Decimals. If we decide to have other decimals we'll keep this in mind.""

OZT-19 | UNLOCKED COMPILER VERSION

Category	Severity	Location	Status
Coding Style	Informational	contracts/TokenPresale.sol (Base): 6	Acknowledged

Description

The contracts cited have an unlocked compiler version. An unlocked compiler version in the source code of the contract permits the user to compile it at or above a particular version. This, in turn, leads to differences in the generated bytecode between compilations due to differing compiler version numbers. This can lead to ambiguity when debugging, as compiler specific bugs may occur in the codebase that would be hard to identify over a span of multiple compiler versions rather than a specific one.

Recommendation

We recommend locking the compiler version to the intended version at deployment.

Alleviation

[Ozak.ai, 06/11/2025]: "Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement."

OPTIMIZATIONS | OZAK.AI - TOKENPRESALE

ID	Title	Category	Severity	Status
OZT-01	Unnecessary Events	Code Optimization	Optimization	Acknowledged

OZT-01 | UNNECESSARY EVENTS

Category	Severity	Location	Status
Code Optimization	Optimization	contracts/TokenPresale.sol (Base): 235, 243	Acknowledged

Description

The OZ library already emits `Paused` and `Unpaused` that can be used in place of the `PresalePaused` and `PresaleResumed` events.

Recommendation

We recommend removing the unnecessary events and using the libraries emitted events.

Alleviation

[CertiK, 06/16/2025]: The client acknowledged the issue and decided to remain unchanged in the scope of the audit.

APPENDIX | OZAK.AI - TOKENPRESALE

Finding Categories

Categories	Description
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.

Checksum Calculation Method

The "Checksum" field in the "Audit Scope" section is calculated as the SHA-256 (Secure Hash Algorithm 2 with digest size of 256 bits) digest of the content of each file hosted in the listed source repository under the specified commit.

The result is hexadecimal encoded and is the same as the output of the Linux "sha256sum" command against the target file.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevating Your Entire Web3 Journey

Founded in 2017 by leading academics in the field of Computer Science from both Yale and Columbia University, CertiK is a leading blockchain security company that serves to verify the security and correctness of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative tech, we're able to support the success of our clients with best-in-class security, all whilst realizing our overarching vision; provable trust for all throughout all facets of blockchain.